
NVIDIA
ASU GPU Day 2025
Use GPUs to Accelerate Vector Databases

Agenda

Vector search

Vector databases

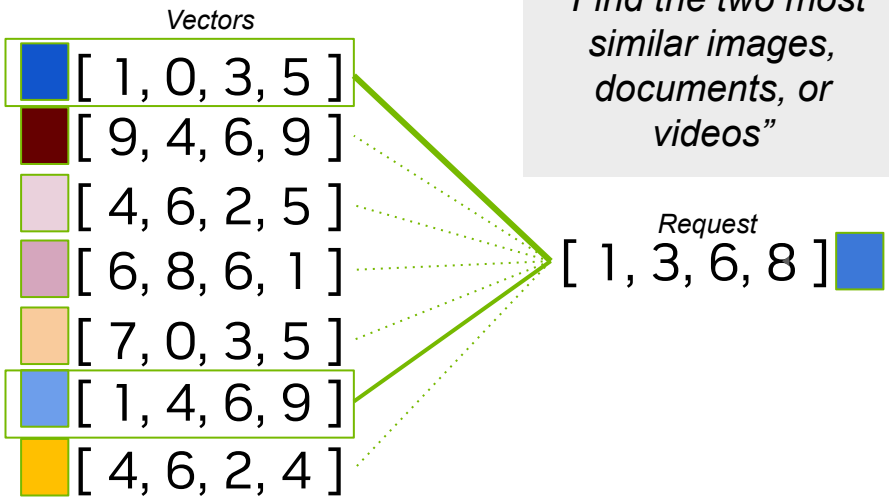
Retrieval Augmented Generation

Intro to vector search

1. What is vector search?
2. Vector search index types
3. Why use GPUs?
4. The NVIDIA cuVS library
5. GPU/CPU Interoperability

What is vector search?

Measuring similarity and retrieving relevant embeddings



Traditional Keyword search

- Matches ***exact words or phrases***:
  Q: Flights with extra legroom
  A: Flights with extra legroom available

Vector search

- Relies on ***distance metrics*** (e.g. cosine, Euclidean) to identify the relevant results.
- Can be used to retrieve results based on ***semantic meaning***:
  Q: Flights with extra legroom
  A: Spacious airline seats

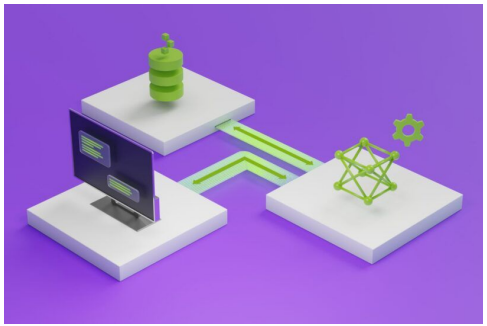
Vector search is a subfield of ***machine learning***!

Where is Vector Search Used?

Semantic Search, Data Mining, and Machine Learning

Semantic search

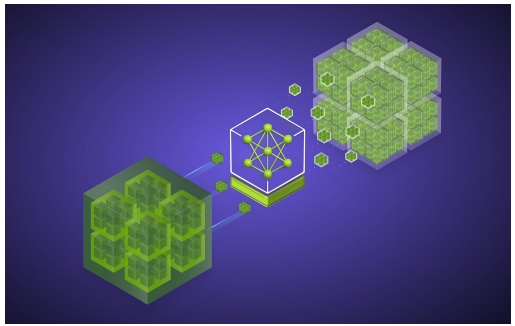
Online, low latency search



- Generative AI & RAG
- Recommender systems
- Molecular search
- Image and video search

Data Mining & ML

Offline, high throughput search



- Knn-graph construction
- Fixing class imbalance
- Clustering
- Dimensionality reduction

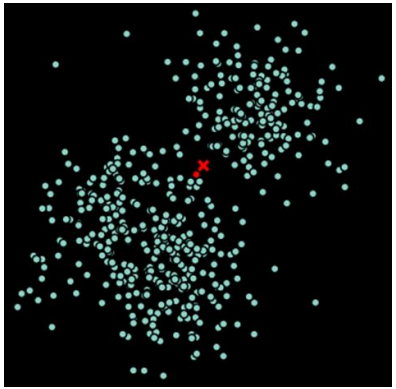
Computing Nearest Neighbors

Naive Exhaustive Search

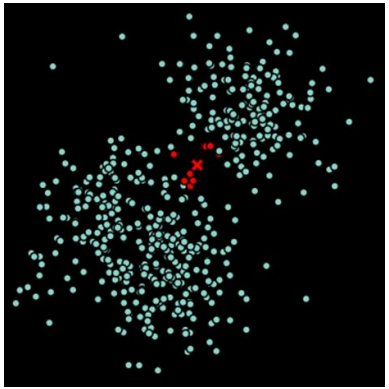
Brute-force: measure the distance to all points and choose the shortest!

This is exhaustive (all n vectors need to be compared against all n vectors) and computationally expensive.

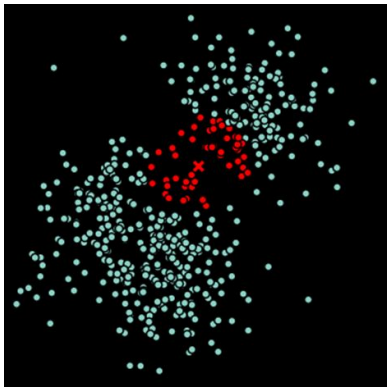
K=1



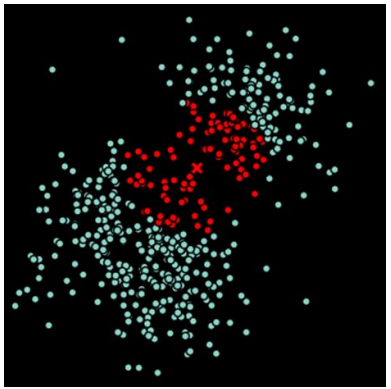
K=10



K=50



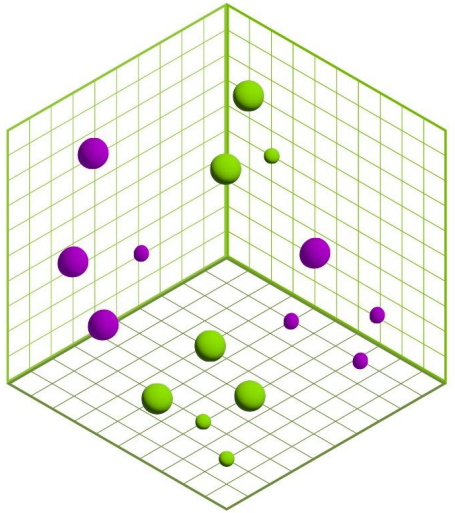
K=100



Approximate Nearest Neighbors

Build a model to approximate the nearest neighbors

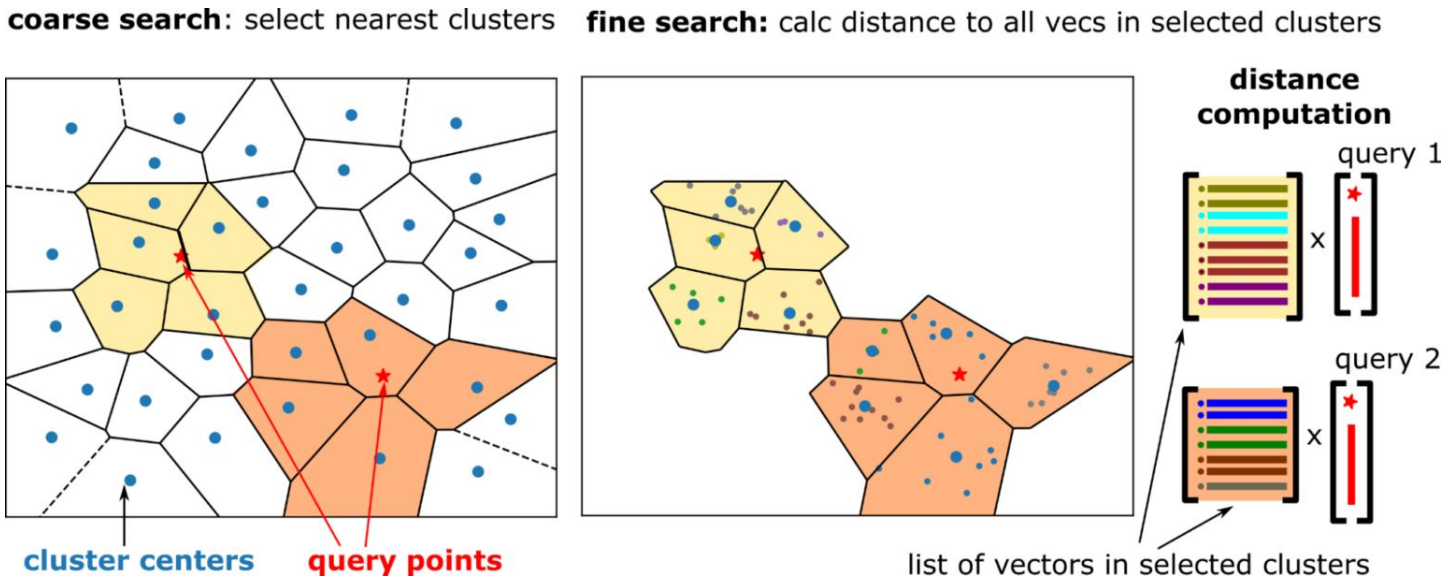
- Brute-force is ***exhaustive and exact***
- Approximate nearest neighbors (ANN) are machine learning models that ***predict the approximate closest neighbors***.
- These models are called indexes and they often take parameters to trade off model ***complexity, quality, and search speed***.



IVF-Flat

Coarse Search and Fine Search

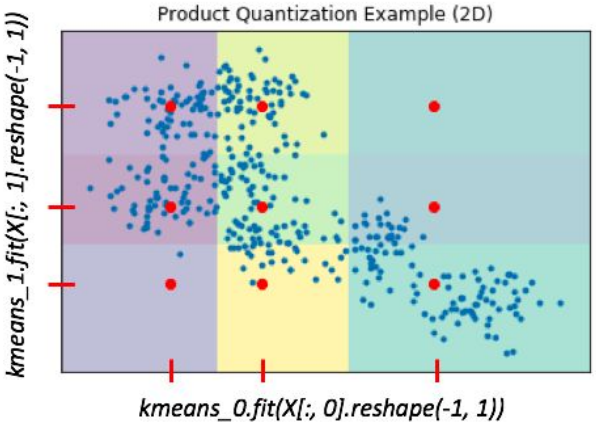
The top-k neighbors from each probed cluster are selected, which results in **n_probes * k_neighbor** candidates for each query. This is reduced to the k-nearest neighbors.

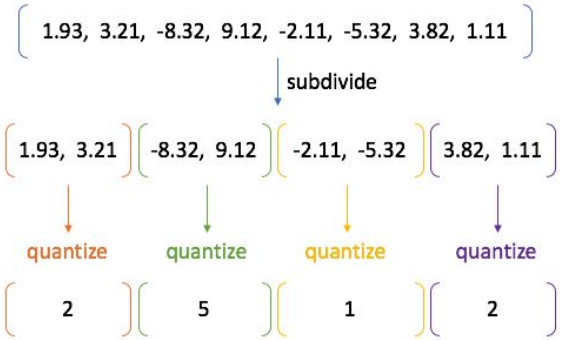


IVF-PQ

Product Quantization

- Lower PQ bits (4-8) provide ***better compression*** and more ***efficient use of shared memory***
- Configurable lookup table and distance precision provide ***faster computation*** and ***efficient use of shared memory***
- Support for ***reduced precision*** (uint8 and int8)
- Supports custom predicate ***pre-filter***
- ***pq_dim*** sets the target dimensionality of the compressed vector.
- ***pq_bits*** sets the number of bits for each vector element after compression.

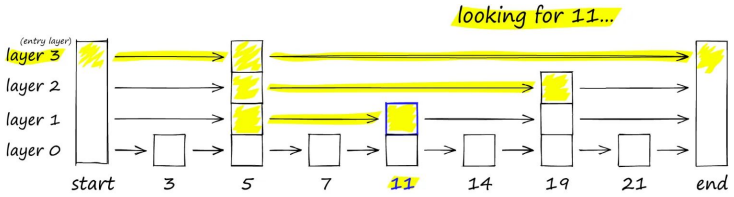




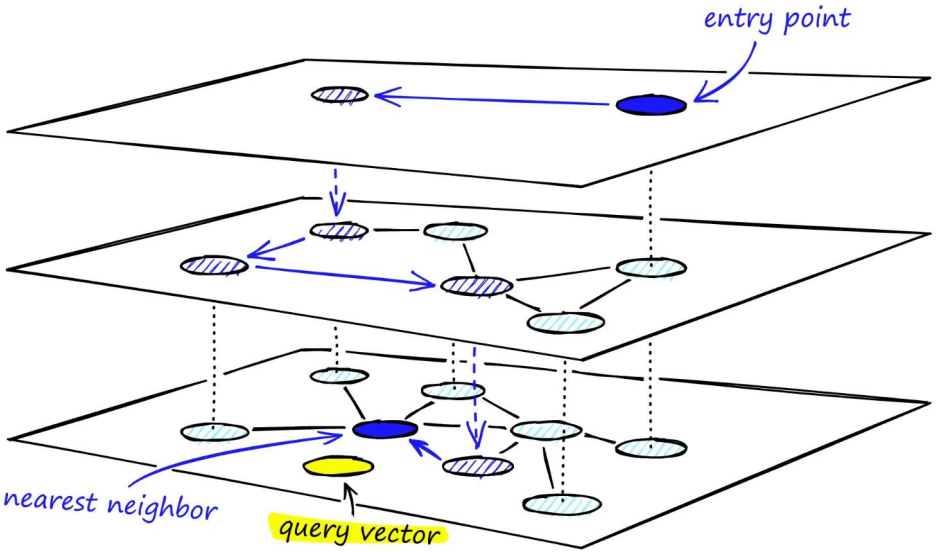
HNSW (CPU only)

Hierarchical Navigable Small World (HNSW) algorithm

Skip List



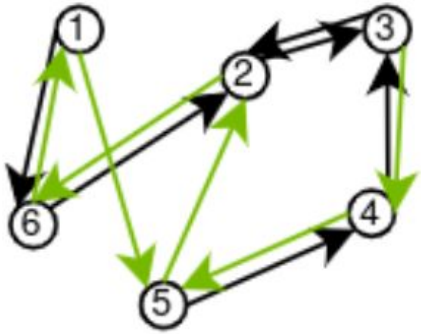
HNSW Graph



CAGRA (GPU only)

GPU accelerated state of the art based ANN

- Open-source, graph-based, ***GPU-native algorithm***
- Parallelizes graph construction, ***lowering build times*** significantly
- Parallelizes ***individual search queries*** resulting in ***high throughput*** especially for ***large batches***
- ***Lowers latency*** for ***online queries***



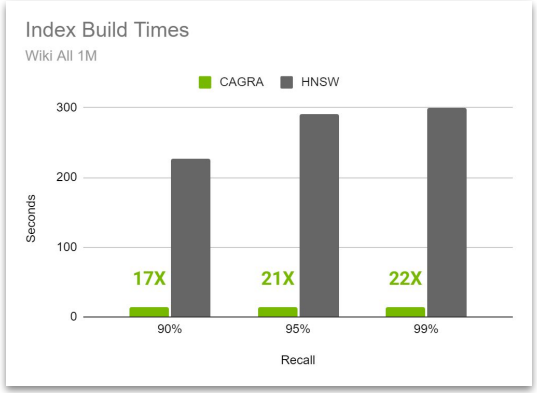
CAGRA graph

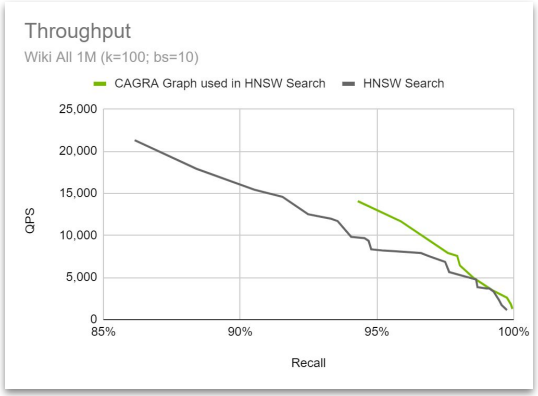
https://arxiv.org/abs/2308.15136

CAGRA and HNSW Interoperability

Building index on GPU and searching on CPU

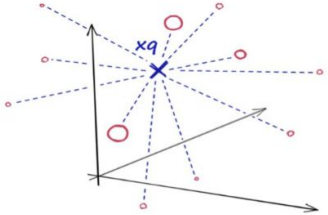
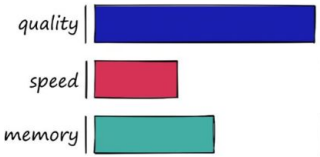
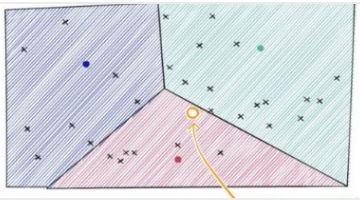
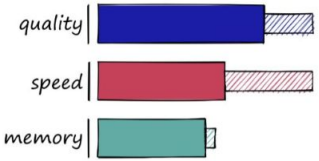
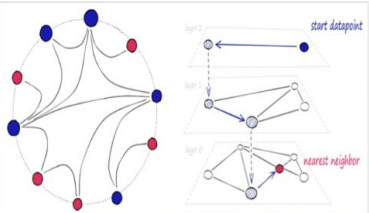
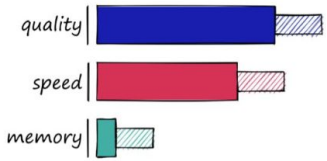
- HNSW ***index building is slow*** – can take hours or days
- CAGRA indexes can be built ***20x faster*** than HNSW
- HNSW can search a graph built with CAGRA
- CAGRA graph can even ***outperform HNSW on CPU search*** at larger dimensions





Comparing Brute Force to ANN Algorithms

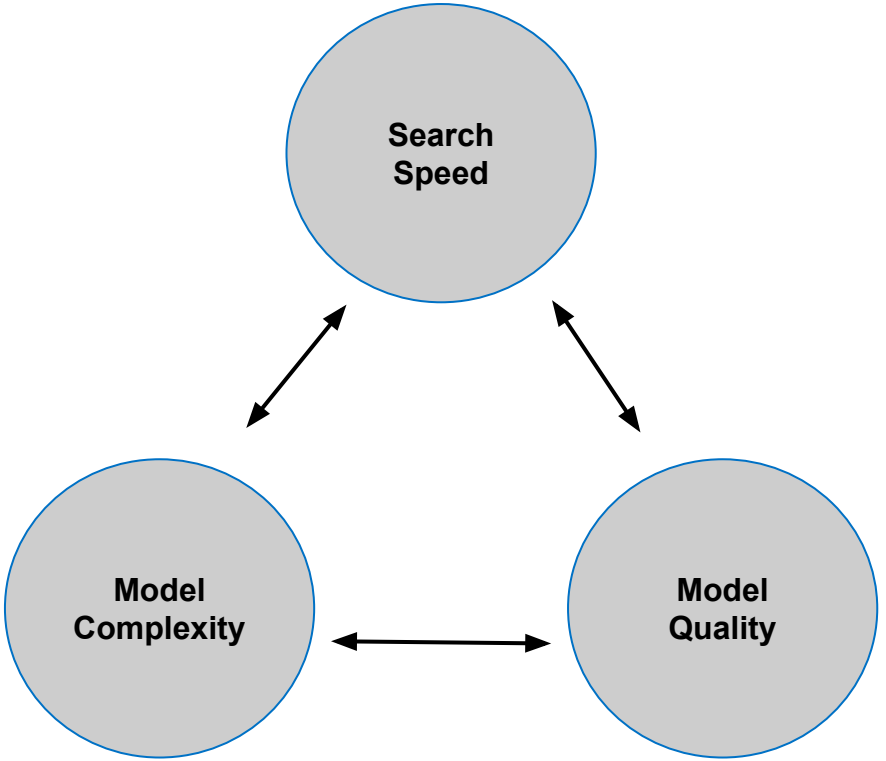
Scaling and speeding up vector search with approximate nearest neighbor methods

		Performance	Characteristics	Examples
Brute Force			Exhaustive search. Exact, but slow – especially at scale	Brute-force
Tree-based			Partitions data to reduce distances during search	IVF SCaNN DiskANN
Graph-based			Constructs a neighborhood graph.	HNSW CAGRA Vamana

Approximate Nearest Neighbors Trade-offs

Competing priorities cause computational challenges and tradeoffs

- ***Higher quality*** search typically requires ***larger storage*** and is more thorough, ***reducing search speed***.
- ***Higher search speeds*** can be achieved with ***lower quality*** models
- Large ***complex models*** can ***increase quality*** at the expense of ***longer build times***, which ***sacrifice speed***.

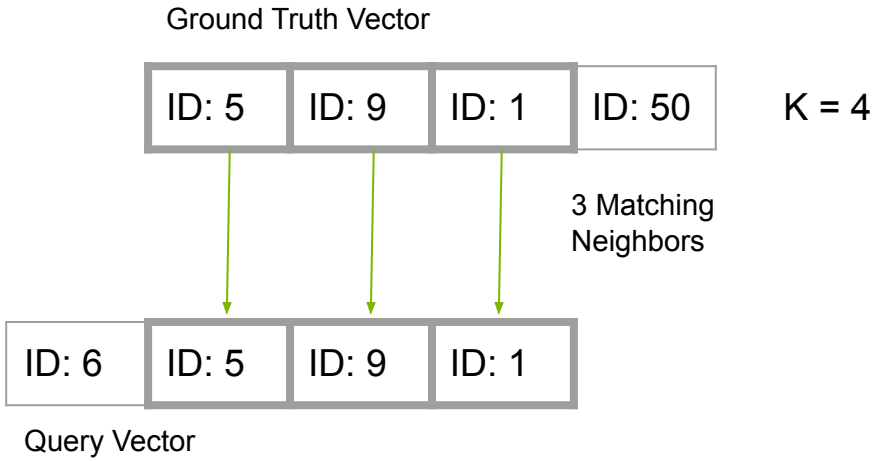


Measuring Quality

Recall = number of correct nearest neighbors found

Measuring quality of an index has 4 steps:

1. Choose some number of query vectors
2. Compute ground truth using exact nearest neighbors for query vectors
3. Train approximate index
4. Search approximate index with query vectors
5. Measure degree of overlap between search result and ground truth



$$\text{Recall} = \frac{\#\ \text{Correct Neighbors}}{K} = \frac{3}{4} = 75\%$$

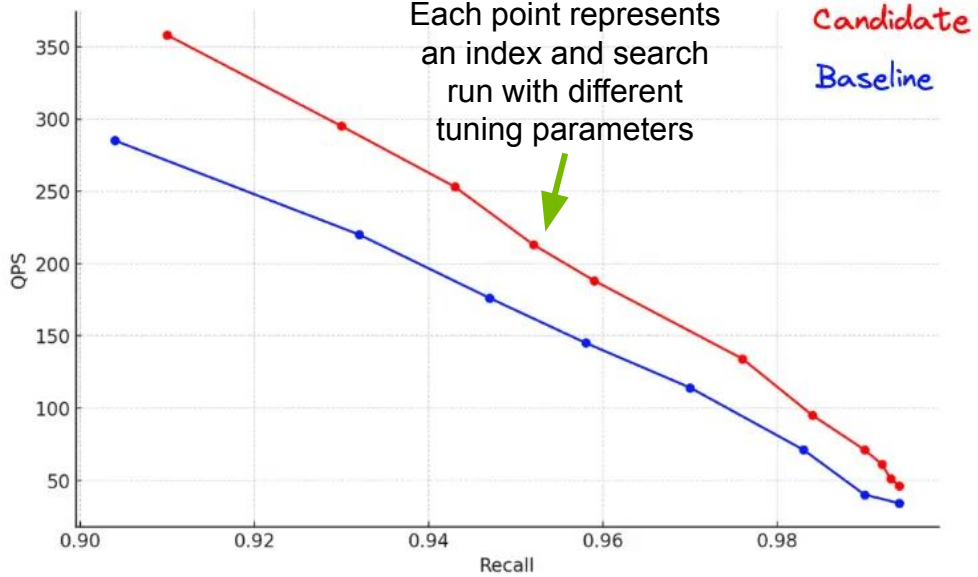
95%-99%
typical recall target

Evaluating Performance

The quality, search speed, and build complexity trade-off

Pareto Front

Search Throughput (or Latency)

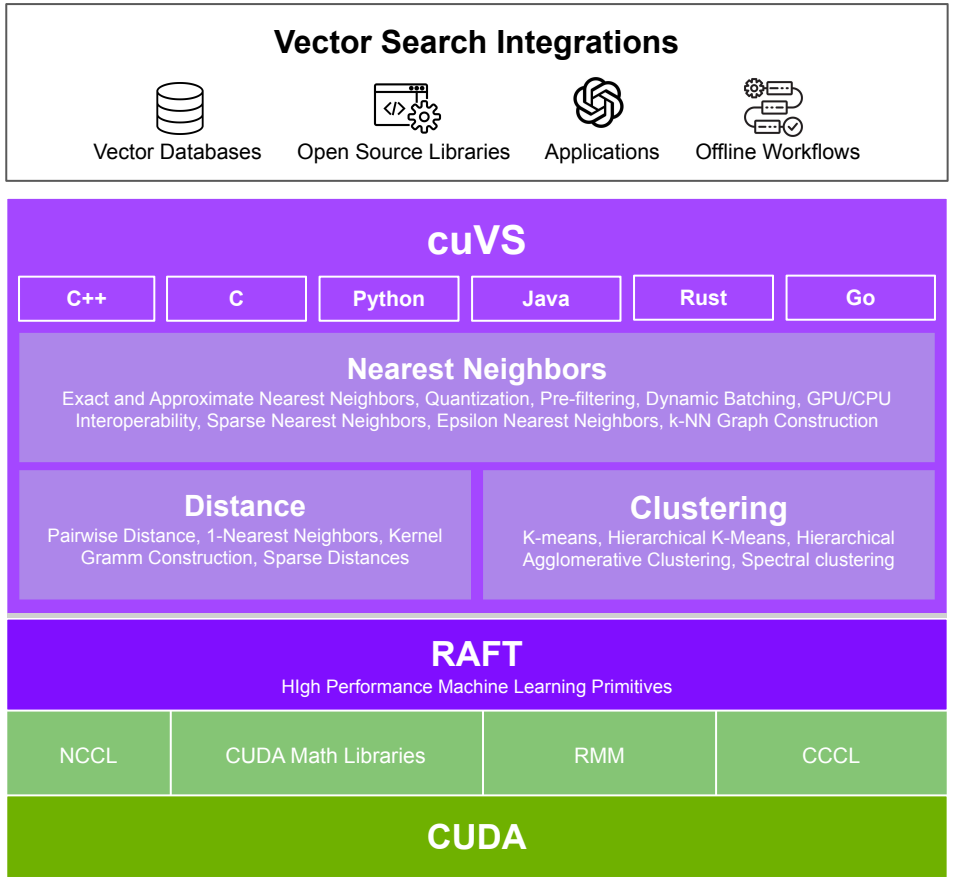


Search Quality

The ***optimal performance*** is evaluated across a ***sweep of build and search parameters***.

NVIDIA cuVS

An Open-source Library for GPU-Accelerated Vector Search



GPUs and Vector Search

Where are the benefits?

Batch/parallel processes run much faster on GPUs

- ANN indexes builds
- Throughput intensive search
- Preprocessing (e.g. quantization)

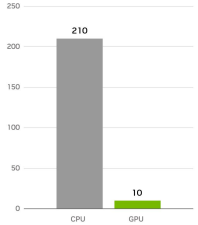
Streaming processes also run faster on GPUs

- Latency sensitive search, especially at high volume

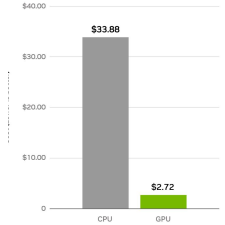
Higher performance can lead to lower costs

- Scaling up index builds that can be deployed to CPU for search

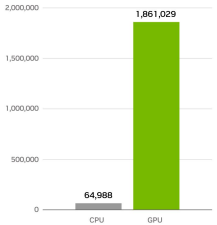
21x Faster Index Build



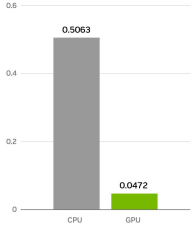
12.5x Lower Index Build Cost



29x Higher Throughput



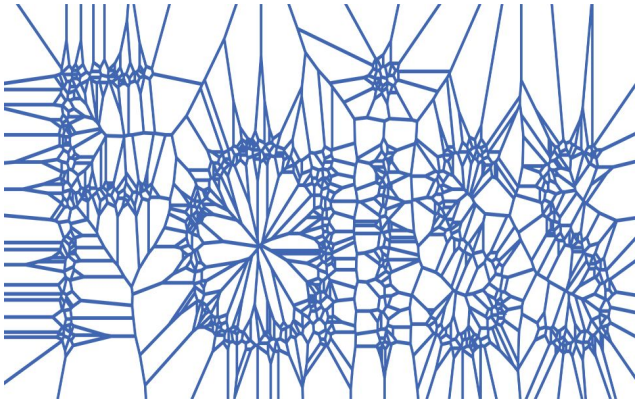
11x Lower Latency



Faiss

Open-source vector search library for CPU and GPU

- Provides ***vector search indexes and preprocessing algorithms*** for both CPU and GPU
- ***Pioneered vector search*** on the GPU
- Indexes are ***interoperable*** (can be converted) between CPU and GPU
- Faiss ***integrates cuVS*** for GPU indexes



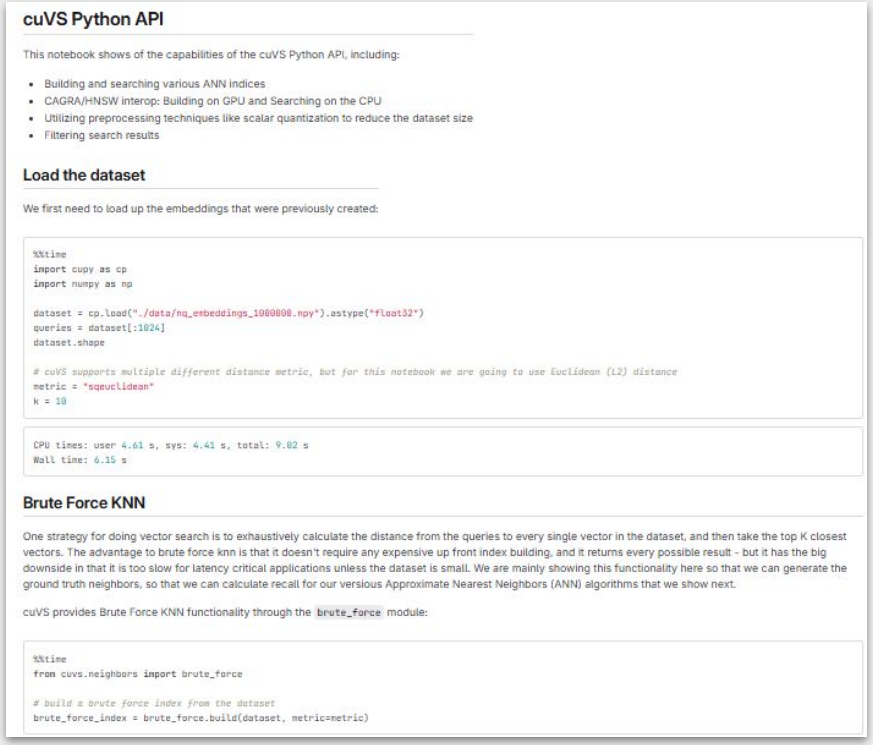
https://github.com/facebookresearch/faiss

Notebook #1

Introduction to vector search with cuVS

This notebook shows of the capabilities of the cuVS Python API, including:

1. Building and searching various ANN indices
2. CAGRA/HNSW interop: Building on GPU and Searching on the CPU
3. Utilizing preprocessing techniques like scalar quantization to reduce the dataset size



cuVS Python API

This notebook shows of the capabilities of the cuVS Python API, including:

- Building and searching various ANN indices
- CAGRA/HNSW interop: Building on GPU and Searching on the CPU
- Utilizing preprocessing techniques like scalar quantization to reduce the dataset size
- Filtering search results

Load the dataset

We first need to load up the embeddings that were previously created:

```
%%time
import cupy as cp
import numpy as np

dataset = cp.load("./data/nq_embeddings_1000000.npy").astype("float32")
queries = dataset[:1024]
dataset.shape

# cuVS supports multiple different distance metric, but for this notebook we are going to use Euclidean (L2) distance
metric = "sqeuclidean"
k = 10
```

```
CPU times: user 4.61 s, sys: 4.41 s, total: 9.02 s
Wall time: 6.15 s
```

Brute Force KNN

One strategy for doing vector search is to exhaustively calculate the distance from the queries to every single vector in the dataset, and then take the top K closest vectors. The advantage to brute force knn is that it doesn't require any expensive up front index building, and it returns every possible result - but it has the big downside in that it is too slow for latency critical applications unless the dataset is small. We are mainly showing this functionality here so that we can generate the ground truth neighbors, so that we can calculate recall for our versious Approximate Nearest Neighbors (ANN) algorithms that we show next.

cuVS provides Brute Force KNN functionality through the `brute_force` module:

```
%%time
from cuvs.neighbors import brute_force

# build a brute force index from the dataset
brute_force_index = brute_force.build(dataset, metric=metric)
```

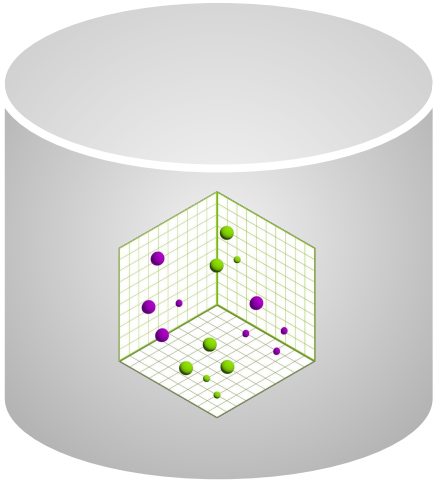
Intro to vector databases

1. What is a vector database?
2. How do databases use vector search?
3. Types of vector databases
4. GPU Accelerated Milvus

What is a vector database?

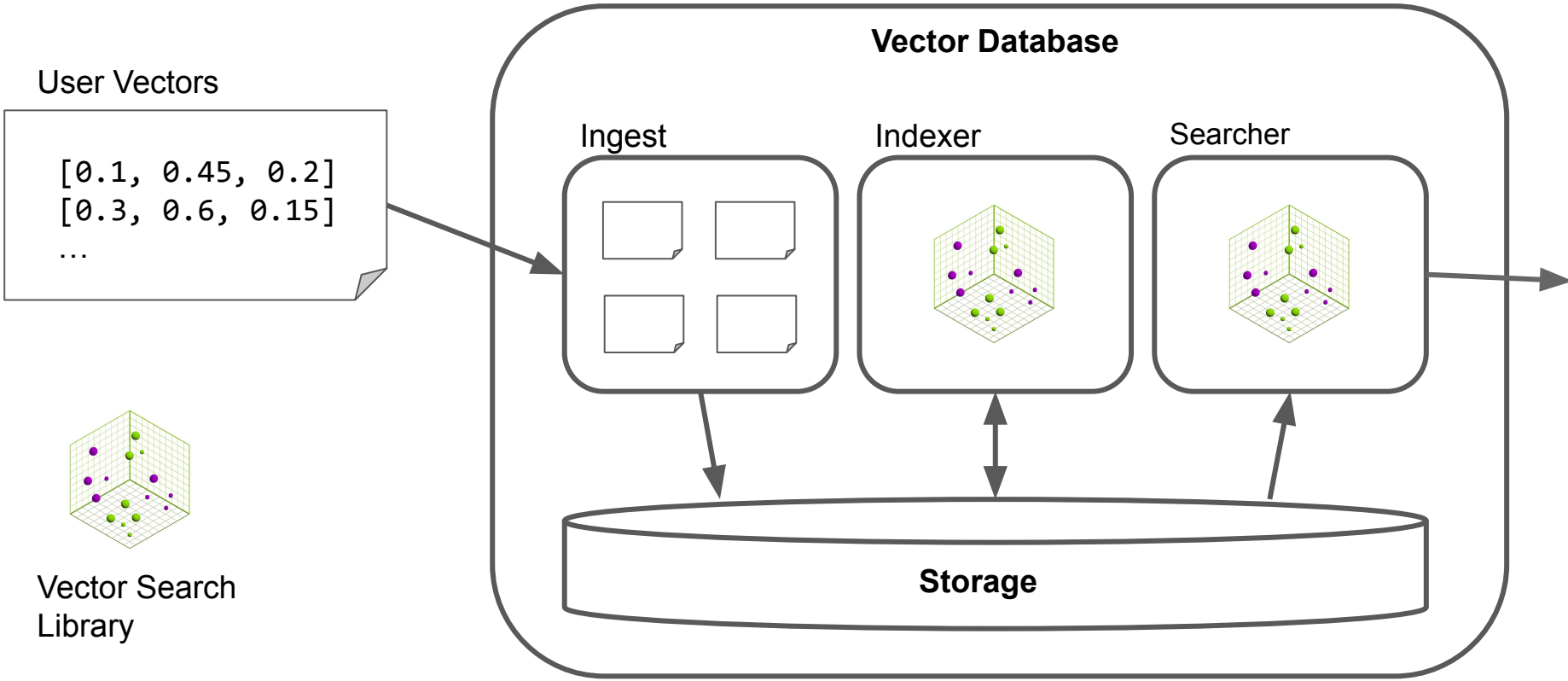
And how does it differ from vector search?

- Vector databases add many guarantees found in traditional databases such as ***scalability, durability, and availability*** for vector search indexes
- They often provide guarantees with regards to ***consistency and freshness*** in the face of ***concurrent ingest and search***.
- They are ***not needed*** for every vector search problem.



How do databases use vector search?

Typical Vector Database Architecture



Types of vector databases

Using Vector Search in Production AI Workflows

Database	Pros	Cons
Special Purpose Weaviate Milvus	• Focus specifically on vector search performance and scale. • Tend to be focused on semantic search and AI	• May not be the only database you'll need for production workflows
Document, Columnar, NoSQL Solr elasticsearch	• Highly scalable (both vertically and horizontally) • Advanced unstructured search capabilities	• Freshness – the ability to query newly ingested data – is often sacrificed for scale • Vector types are a bolt-on feature (today)
Structured Transactional AlloyDB ORACLE PostgreSQL	• Transactional guarantees (immediate freshness) • Optimized for structured search	• Not as horizontally scalable as other database types • Vector types are a bolt-on feature (today)

GPU Accelerated Milvus

A cuVS powered, open-source database for GenAI apps

Milvus is a popular, open-source, special purpose vector database

Developers use Milvus for:

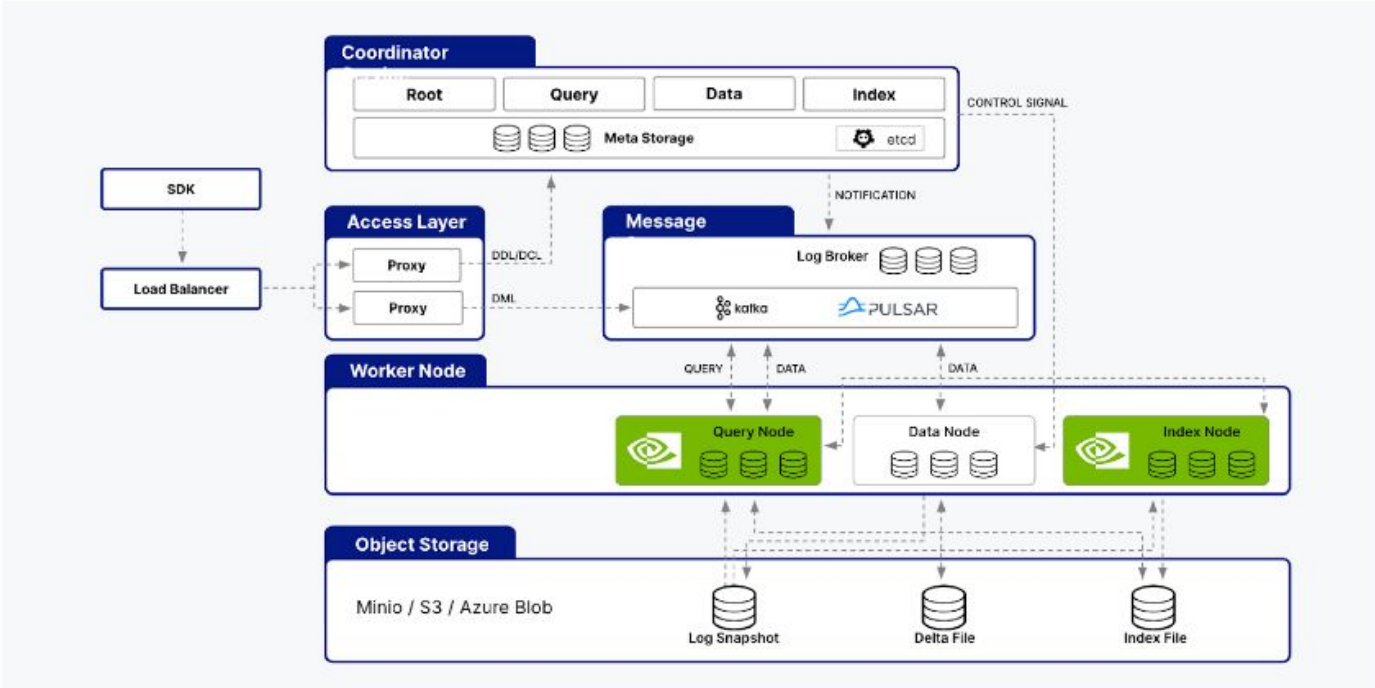
- Performance
- Scaling
- Features
- Open-source community
- Plays nicely with LangChain, LlamaIndex, OpenAI, Hugging Face, etc.

NVIDIA cuVS is part of Milvus

- *Perform High-Efficiency Search, Improve Data Freshness, and Increase Recall With GPU-Accelerated Vector Search and RAG Workflows (GTC 2024)*

GPU Accelerated Milvus

A cuVS powered, open-source database for GenAI apps



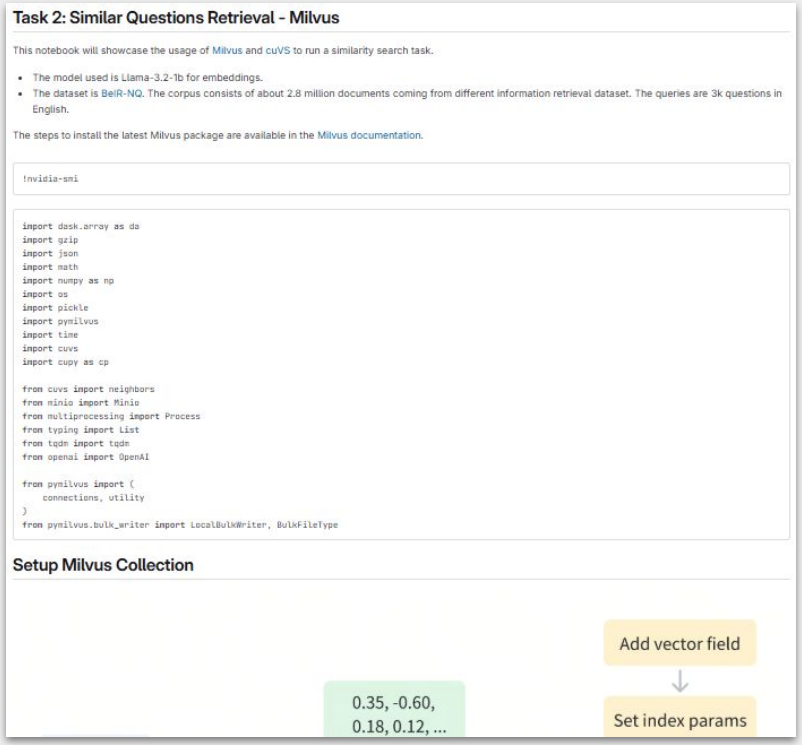
NVIDIA cuVS algorithms are integrated into Milvus 2.2+

Notebook #2

Introduction to vector database with Milvus

This notebook will showcase the usage of Milvus and cuVS to run a similarity search task.

1. The model used is Llama-3.2-1b for embeddings.
2. The dataset is BeIR-NQ. The corpus consists of about 2.8 million documents coming from different information retrieval dataset. The queries are 3k questions in English.



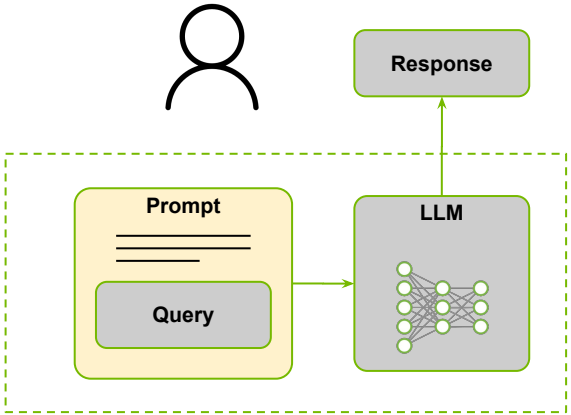
Overview of RAG

1. What is RAG?
2. Typical RAG workflow

Large Language Models

Generative AI without RAG

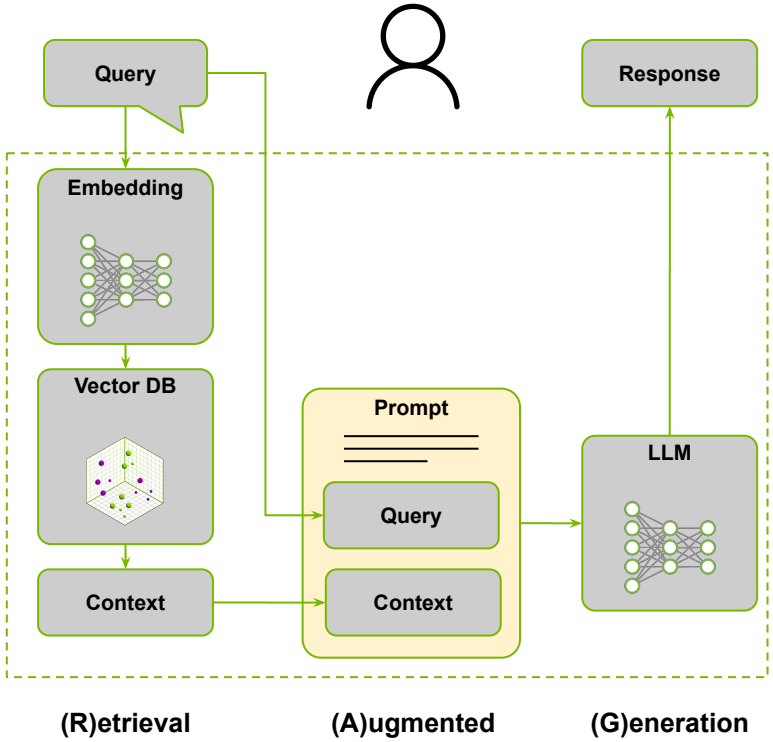
- Accept a passage (of text, image, video, etc…) and predict or ***generate the next word*** in the passage.
- Requires a ***significant amount of compute*** and data to train.
- Can hallucinate when asked about ***topics outside of their training data***, such as current events.



What is RAG?

Retrieval Augmented Generation

RAG allows us to ***utilize vector search*** to perform a ***semantic search*** on passages of data in order to ***give context to LLMs*** on data they were not trained on.



Typical RAG Workflow

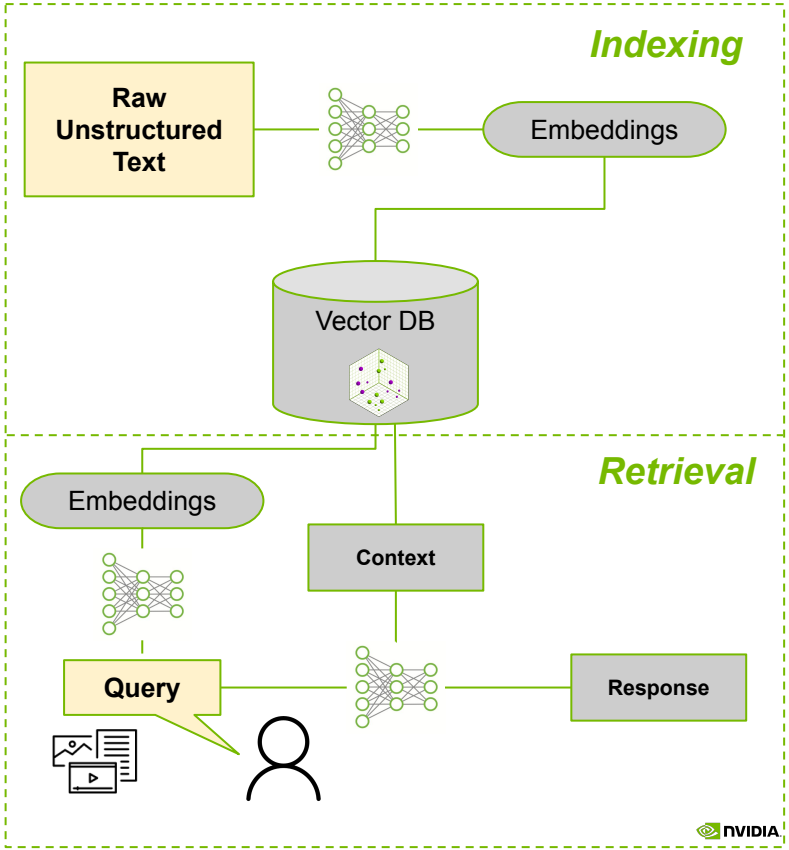
Retrieval Augmented Generation

Indexing - Input: **Raw unstructured data** (text, video, etc.)

1. Create embeddings
   a. Chunk raw input into smaller passages
   b. Run each chunk through embedding model to create vectors
2. Ingest vectors and raw passages into database
3. Build the index

Retrieval - Input: **Query** (text, video, etc.)

1. Create embeddings
   a. Chunk prompt into smaller passages
   b. Create embeddings from each passage
2. Perform vector search for passages
3. Get raw passages for closest vectors
4. Run passages through LLM to generate outputs



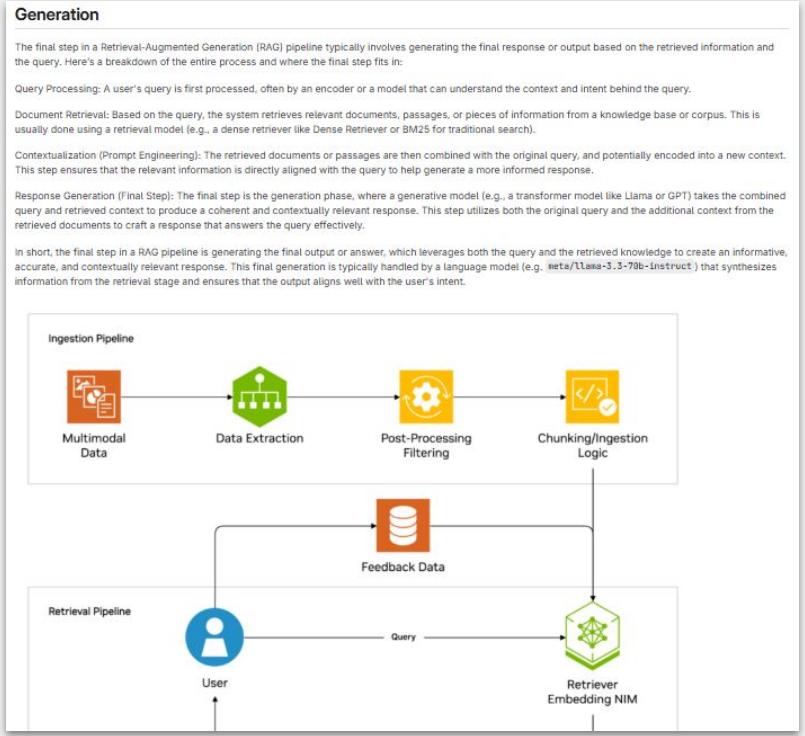
Notebook #3

RAG Generative Step

The final step in a RAG pipeline is generating the final output or answer, which leverages both the:

1. Query and
2. Retrieved knowledge

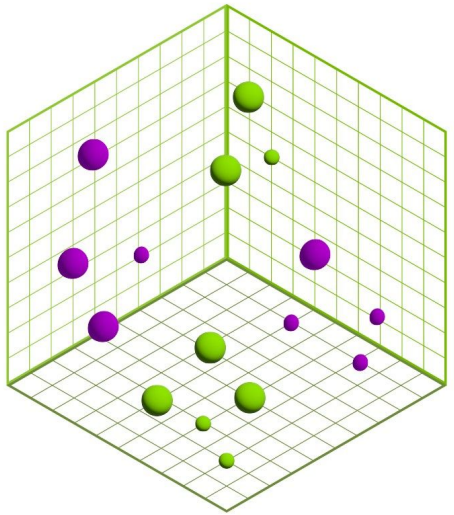
To create an informative, accurate, and contextually relevant response.



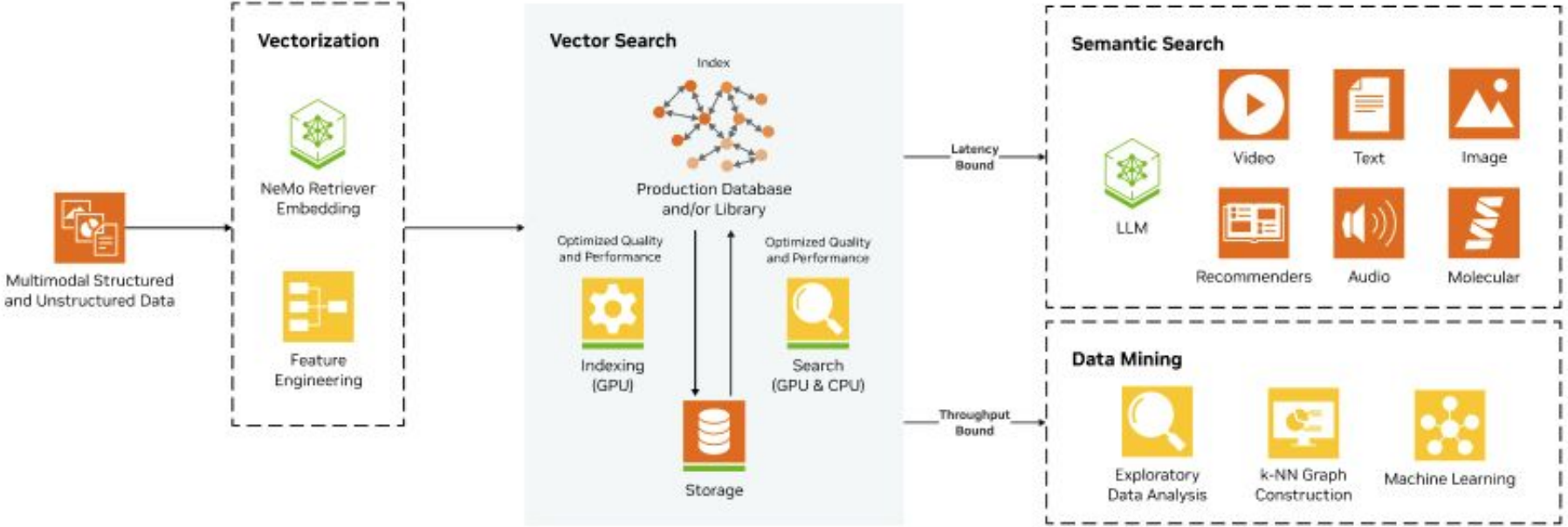
Summary

Vector Search Summary

1. Vector search is a subfield of ***machine learning***
2. Approximate models ***speed up nearest neighbors*** at the cost of correctness
3. Recall is used to ***evaluate ANN models***
4. GPUs accelerate ***index construction and search*** and can ***lower costs***.
5. cuVS is a ***library for vector search*** on the GPU
6. cuVS indexes can be built on the GPU and ***converted to the CPU*** for search



How NVIDIA cuVS Works



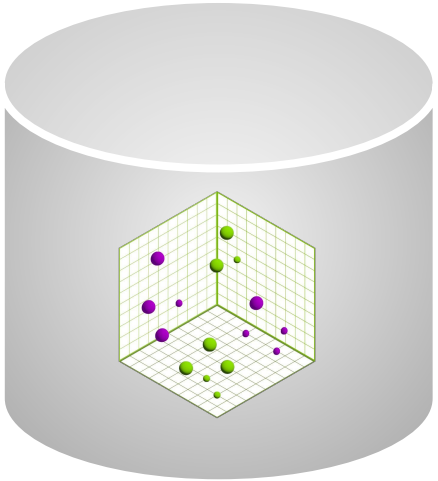
Vector Database Summary

1. Vector databases add many ***systems-level guarantees*** found in traditional databases.

2. Vector databases typically rely on ***vector search indexes***

3. Nearly all types of databases are ***incorporating vector types***, but there are trade-offs to each.

4. While many vector databases are ***adopting GPUs through the cuVS library***, this lab is focused on Milvus.



RAG Summary

1. Non-RAG LLM workflows can ***suffer from hallucinations***

2. RAG ***adds a semantic search step*** to provide context to LLM workflows